

Refining HTN Methods via Task Insertion with Preferences (ArXiv Version)

Zhanhao Xiao,¹ Hai Wan,^{1*} Hankui Hankz Zhuo,^{1†} Andreas Herzig,^{2,3} Laurent Perrussel,^{2,3} Peilin Chen¹

¹School of Data and Computer Science, Sun Yat-sen University, Guangzhou, China

²IRIT, Toulouse, France

³University of Toulouse, Toulouse, France

Abstract

Hierarchical Task Network (HTN) planning is showing its power in real-world planning. Although domain experts have partial hierarchical domain knowledge, it is time-consuming to specify all HTN methods, leaving them incomplete. On the other hand, traditional HTN learning approaches focus only on declarative goals, omitting the hierarchical domain knowledge. In this paper, we propose a novel learning framework to refine HTN methods via task insertion with completely preserving the original methods. As it is difficult to identify incomplete methods without designating declarative goals for compound tasks, we introduce the notion of prioritized preference to capture the incompleteness possibility of methods. Specifically, the framework first computes the preferred completion profile *w.r.t.* the prioritized preference to refine the incomplete methods. Then it finds the minimal set of refined methods via a method substitution operation. Experimental analysis demonstrates that our approach is effective, especially in solving new HTN planning instances.

Introduction

Hierarchical task network (HTN) planning techniques (?) are increasingly used in a number of real-world applications (?; ?). In the real-world logistics domain, such as Amazon and DHL Global Logistics, the shipment of packages is arranged via decomposition into a more detailed shipment arrangement in a top-down way according to the predefined HTN methods. In practice, there exist a vast number of cases occurring, such as the delay caused by the weather, leading that it is difficult and time-consuming for humans to find all complete methods for all actions. This suggests that it is important to learn complete methods.

Normally, domain experts have partially hierarchical domain knowledge, which possibly is not sufficient to cover all desirable solutions (?). On one hand, with partially hierarchical domain knowledge, a planner may fail to obtain

a solution via decomposition according to the given methods. The main reason lies in that the given method set is incomplete, which includes at least an incomplete method lacking subtasks. On the other hand, the hierarchical domain knowledge comes from the experience and investigation of domain experts, which normally are supposed to be necessary. However, the traditional approaches to learning HTN methods, such as (?), only concentrate on declarative goals of compound tasks and omit hierarchically procedural knowledge obtained from domain experts. Indeed, these procedural knowledge cannot be expressed by only operator structures (action models), which can be concluded from (?; ?). Therefore, in this paper, we focus on refining HTN methods and keeping the original hierarchical domain knowledge from domain experts.

To tackle the method incompleteness, Geier and Bercher (?) proposed a hybrid planning formalization, *HTN planning with task insertion (TIHTN planning)*, to allow generating plans via decomposing tasks according to the methods but also inserting tasks from outside the given methods. Example 1 shows a plan with task insertion (called TIHTN plan) for an HTN problem with incomplete methods. Actually, the TIHTN plan offers a reference to refine the methods by adding the inserted tasks. For example, the method of *airShip* can be refined by adding *fly* as its subtask. It allows us to absolutely preserve the original domain knowledge to refine the incomplete methods, further to assist domain experts to improve the HTN domain.

Example 1. Consider an example in the logistics domain, suppose every task has only one method and a decomposition tree is shown in Figure 1. The initial task *ship(pkg1, whA, shopB)* is to ship a package from city A to city B and it has a method: to ship the package from the warehouse to the airport by truck, from city A to city B by plane and from the airport to the shop by truck. But in case that the plane is not in the airport of city A, the air transportation task *airShip(pkg1, airpA, airpB)* cannot be accomplished, neither can the initial task. When arranging the plane to airport A, *fly(plane1, airpA)*, is done before loading to the plane, an executable plan is found.

Whereas, even if a TIHTN plan is found, without designating declarative goals for compound tasks, it is still dif-

*Corresponding author

†Key Laboratory of Machine Intelligence and Advanced Computing, Ministry of Education, China; Guangdong Province Key Laboratory of Big Data Analysis and Processing, China.
Copyright © 2020, Association for the Advancement of Artificial Intelligence (www.aaai.org). All rights reserved.

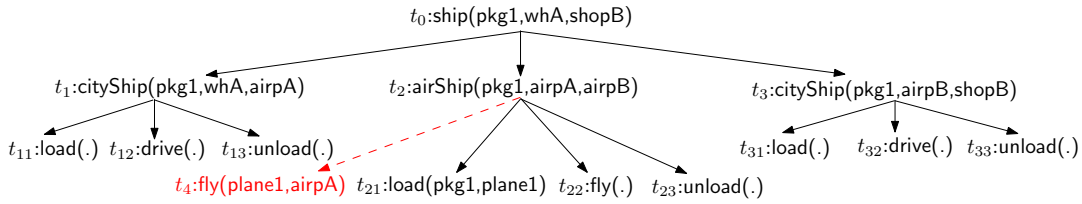


Figure 1: An example of a decomposition tree from incomplete methods (the parameters of some actions are hidden). The initial task ship(pkg1, whA, shopB) is decomposed into a sequence of primitive tasks (the black leaves) according to the original methods. But when plane1 is not in airport A, the sequence is not executable. It becomes executable if arranging plane 1 to airport A before loading the package, which implies that fly(plane1, airpA) should be considered as a subtask of airShip.

difficult to identify incomplete methods – because an inserted task can be considered as a missed subtask of different methods. For example, the inserted task fly can also be used to refine the method of cityShip. In practice, the missing of subtasks happens more likely on certain methods than on some other methods. It motivates us to introduce the notion of prioritized preference on methods to capture the incompleteness possibility of methods.

Our contributions are listed as follows. First, we introduce the notion of completion profiles to refine HTN methods from TIHTN plans. Second, we propose a framework METHODREFINE to refine HTN methods with completely preserving the original methods. Specifically, we first compute the preferred completion profile *w.r.t.* the prioritized preference to refine incomplete methods and then propose a method substitution operation to obtain the preferred set of refined methods. Third, by taking experiments on three well-known planning domains, we compare our approach with different method incompleteness against the classical HTN learning approach, HTN-MAKER, on the ability of solving new instances in the same domain and show that our approach is more effective.

Related Work

Besides HTN-MAKER, there are a number of HTN learning approaches (e.g., ?) which only focus on declarative goals and omit hierarchically procedural knowledge. Specially, they require annotated preconditions and effects of compound tasks and only consider the declarative goals like classical planning, so they require a complete executable plan as input. Whereas, in many domains, it is difficult to verify the correctness of the task annotations. Besides, these approaches restrict the tasks in the methods to be totally ordered, while we allow them to be partially ordered. Last but not least, comparing with these approaches, we also consider the prioritized preference on the learned methods.

Another related work is (?) which proposes an approach to construct and maintain hierarchical task models from a set of annotated examples provided by domain experts. Similar to the annotated tasks, obtaining these annotated examples is difficult and needs a lot of human effort. Our work also is related to the works on learning the precondition of HTN methods (e.g., ?), which takes the hierarchical relationships between tasks, the action models, and a complete description of the intermediate states as input. The similar work also in-

cludes (?) and (?), which used means-end analysis to learn structures and preconditions of the input plans. The precondition and effect of primitive actions can also be learned in (?). All these approaches of learning method precondition require a complete method set as input.

For TIHTN planning, Geier and Bercher (?) first addressed the decidability of propositional TIHTN planning. Later Alford *et al.* (?) proved that propositional TIHTN planning is EXPTIME-complete and proposed an acyclically searching approach to compute TIHTN plans, which provides a route to refine incomplete HTN methods.

Definitions

We adapt the definitions of lifted HTN planning (?). First, we define a function-free first order language $\mathcal{L}$ from a set of variables and a finite set $\mathcal{L}_0$ of predicates and constants. We take parts of variables in $\mathcal{L}$ as *task symbols* to identify *tasks*. A state is any subset of ground atoms in $\mathcal{L}$ and the finite set of states are denoted by $2^{\mathcal{L}}$. In HTN planning, actions (or task names), noted $\mathcal{A}$, are syntactically first-order atoms, which are classified into two categories: the actions the agent can execute directly are called *primitive actions* or *operators*, noted $\mathcal{O}$, while the rest are called *compound actions*, noted $\mathcal{C}$. Every primitive action o is a tuple (name(o), pre(o), add(o), del(o)) where name(o) consists of a predicate out of $\mathcal{L}$ and a list of variables, called its name; pre(o) is a first-order logic formula, called its precondition; add(o) and del(o) are two conflict-free sets of atoms called its positive and negative effect. Then we define a state-transition function $\gamma : 2^{\mathcal{L}} \times \mathcal{O} \rightarrow 2^{\mathcal{L}}$: $\gamma(s, o)$ is defined if o is *applicable* in s (i.e., $s \models \text{pre}(o)$); $\gamma(s, o) = (s \setminus \text{del}(o)) \cup \text{add}(o)$. A sequence of primitive actions $o_1, \dots, o_n$ is *executable* in a state s_0 iff there is a state sequence $s_1, \dots, s_n$ such that $\forall 1 \leq i \leq n, \gamma(s_{i-1}, o_i) = s_i$.

Given a set R , we use $\vec{R}$ to denote the set of all sequences over R and use $|R|$ to denote the cardinality of R . For its subset X and a function $f : R \rightarrow S$, its restriction to X is $f|_X = \{(r, s) \in f \mid r \in X\}$. For a binary relation $Q \subseteq R \times R$, we define its restriction to X by $Q|_X = Q \cap (X \times X)$.

Task networks. A task network is a tuple $\text{tn} = (T, \prec, \alpha)$ where T is a set of tasks, $\prec \subseteq T \times T$ is a non-empty set of ordering constraints over T and $\alpha : T \rightarrow \mathcal{A}$ labels every task with an action. Every task is associated to an action and the ordering constraints restrict the execution order of tasks. A

task t is called *primitive* if $\alpha(t)$ is primitive (otherwise called *compound*), and called *ground* if $\alpha(t)$ is ground. A task network is called primitive if it contains only primitive tasks, and called ground if it contains only ground tasks.

HTN methods. Compound actions cannot be directly executed and need to be decomposed into a task network according to HTN methods. An *HTN method* $m = (c, \text{tn}_m)$ consists of a compound action c (called *head*) and a task network tn_m whose inner tasks are called *subtasks*. Generally, an HTN method includes variables, which can be grounded as actions are grounded. Note that a compound action c may have more than one HTN method.

Intuitively, decomposition is done by selecting a compound task, adding its subtask network and replacing it. The constraints about the decomposed task t are propagated to its subtasks: the tasks before t are before all its subtasks and the tasks after t are after all its subtasks.

A task network $\text{tn} = (T, \prec, \alpha)$ is a grounding of another task network $\text{tn}' = (T', \prec', \alpha')$ if there exists a bijection $f: T \rightarrow T'$ such that $\alpha(t)$ is a grounding of $\alpha'(f(t))$ and for all $t_1 \prec t_2$, $f(t_1) \prec' f(t_2)$.

HTN problems. An HTN planning domain is a tuple $\mathfrak{D} = (\mathcal{L}, \mathcal{O}, \mathcal{C}, \mathcal{M})$ where $\mathcal{M}$ is a set of HTN methods and $\mathcal{O} \cap \mathcal{C} = \emptyset$. We call a pair (s_0, t_0) an *instance* where s_0 is a ground initial state and t_0 is a ground initial task. An HTN problem is a tuple $\mathcal{P} = (\mathfrak{D}, s_0, t_0)$.

Solutions. A solution to an HTN problem $\mathcal{P} = (\mathfrak{D}, s_0, t_0)$ is a valid decomposition tree $\mathcal{T}$ w.r.t. $\mathcal{P}$ and we say (s_0, t_0) is solved under $\mathfrak{D}$ and is satisfied by $\mathcal{T}$.

In different literature, the solution to the HTN problem has different forms: mostly a plan (such as (?)), a primitive task network (such as (?)) and a list of decomposition trees (such as (?)). In this paper, we consider a solution to the HTN problem as a decomposition tree rooted in the initial task t_0 .

A decomposition tree (DT) is a tuple $\mathcal{T} = (T, E, \prec, \alpha, \beta)$ where (T, E) is a tree, with nodes T and with directed edges $E: T \rightarrow \vec{T}$ mapping each node to an ordered list of its children; $\prec$ is a set of constraints over T ; function $\alpha: T \rightarrow \mathcal{A}$ links tasks and actions; function $\beta: T \rightarrow \mathcal{M}$ labels every inner node with an HTN method.

We use $\ll$ to denote the transitive closure of $\prec$ and the order defined by E . We say t_1 is a predecessor of t_2 if $t_1 \ll t_2$. Dually, we also say t_2 is a successor of t_1 . According to $\ll$, we say the sequence constituted by the ground leaf nodes of $\mathcal{T}$ is its plan, denoted by $\vartheta(\mathcal{T})$.

Definition 1 (Valid DTs). A DT $\mathcal{T}$ is valid w.r.t. an HTN problem $\mathcal{P} = (\mathfrak{D}, s_0, t_0)$ iff its plan $\vartheta(\mathcal{T})$ is executable in s_0 and its root is t_0 and for every inner node t where $\beta(t) = (c, \text{tn}_m)$, it satisfies:

1. $\alpha(t) = c$;
2. $(E(t), \prec|_{E(t)}, \alpha|_{E(t)})$ is a grounding of tn_m ;
3. if $(t, t') \in \prec$ then for every $st \in E(t)$, $(st, t') \in \prec$;
4. if $(t', t) \in \prec$ then for every $st \in E(t)$, $(t', st) \in \prec$;
5. there are no t_1, t_2 such that $t_1 \ll t_2$ and $t_2 \ll t_1$.

Example 2 (Example 1 cont.). If *plane1* is already at airport A in s_0 , the DT drawn with **black** arrows in Figure 1 is a solution to the HTN problem with a plan $\sigma_1 = \langle \text{load}; \text{drive}; \text{unload}; \text{load}; \text{fly}; \text{unload}; \text{load}; \text{drive}; \text{unload} \rangle$.

Refining Methods via Task Insertion

In this paper, we focus on the HTN problem with an incomplete method set, where there is no valid decomposition tree w.r.t. the problem. In other words, there is no executable plan obtained only by applying methods. By allowing inserting tasks, (?) proposes a hybrid planning formalization, TIHTN planning. For an HTN problem $\mathcal{P} = (\mathfrak{D}, s_0, t_0)$, we say a primitive action sequence σ is it TIHTN plan, if σ is executable in s_0 and there is a valid DT $\mathcal{T}$ w.r.t. $\mathcal{P}$ whose $\vartheta(\mathcal{T})$ is not required to be executable in s_0 satisfying $\vartheta(\mathcal{T})$ is a sub-sequence of σ .

Example 3 (Example 2 cont.). If *plane1* is not at airport A in s_0 , the DT in Example 2 is not valid as its plan σ_1 is not executable in s_0 . While $\sigma_2 = \langle \text{load}; \text{drive}; \text{unload}; \text{fly}; \text{load}; \text{fly}; \text{unload}; \text{load}; \text{drive}; \text{unload} \rangle$ is a TIHTN plan to the problem.

Refining Methods and Completing DTs

Actually, the inserted tasks in the TIHTN plan are subtask candidates: they provide clues for refining the original methods by adding them as subtasks. Then, based on a TIHTN plan, we propose the completion profile to refine methods and complete decomposition trees.

Suppose the TIHTN planner outputs a plan σ and its corresponding decomposition tree $\mathcal{T}$, we use I_σ to denote all the inserted tasks in σ . The TIHTN plan actually is an ordering of primitive tasks and we extend the $\ll$ relation of $\mathcal{T}$ by considering the execution order of primitive actions in σ . To get the compound tasks, we use $N_\mathcal{T}$ to denote the inner nodes of $\mathcal{T}$. Next, we show how to link these inserted tasks with the inner nodes $N_\mathcal{T}$ of the decomposition tree $\mathcal{T}$ to generate a new decomposition tree.

Definition 2. We define a completion profile as a function $\rho: I_\sigma \rightarrow N_\mathcal{T}$, such that for every inserted task $t' \in I_\sigma$ there is not a primitive task $t_p \in \sigma$ where either both $t_p \ll \rho(t')$ and $t' \ll t_p$, or $\rho(t') \ll t_p$ and $t_p \ll t'$.

Intuitively, every inserted task is associated with a compound task as its subtask. Every inserted task is restricted to be performed before the predecessors and after the successors of its corresponding compound task.

Next, we define how to refine a method by inserting tasks. A completion profile leads to a set of refined methods by adding the relevant inserted tasks into the original methods. Formally, for a completion profile ρ and $t \in N_\mathcal{T}$, we use $T_\rho^t = \{t' \mid \rho(t') = t\}$ to denote all inserted tasks associated with t . The inserted subtasks with the original subtasks of t compose a new subtask network, written by $\text{tn}_\rho^t = (T_\rho^t, \ll|_{T_\rho^t}, \alpha_\sigma)$, where α_σ which labels the inserted task with ground primitive actions. Suppose $m = (c, (T_m, \prec_m, \alpha_m))$ is the method of t , i.e., $\beta(t) = m$, for every constant in the inserted primitive actions which occurs in the ground actions associated with the children of t or $\alpha(t)$, we

replace it with its corresponding variable in the unground action in tn_m and update the function α_σ to α'_σ . Then we define the refined method of m as $m_\rho^t = (c, (T_m \cup T_\rho^t, \prec_m \cup \ll_{|T_m \cup T_\rho^t, \alpha_m \cup \alpha'_\sigma}))$ w.r.t. ρ . We use $\mathcal{M}_\rho$ to denote the set of refined methods from the completion profile ρ .

Example 4 (Example 3 cont.). We have a completion profile ρ where $\rho(t_4) = t_2$. The refined method of the original method m is $m_\rho^{t_2} = (\text{airShip}(\text{?pkg}, \text{?loc1}, \text{?loc2}), (T', \prec', \alpha'))$ where

- $T' = \{t'_4, t'_{21}, t'_{22}, t'_{23}\}$;
- $\prec' = \prec_m \cup \{(t'_4, t'_{21}), (t'_4, t'_{22}), (t'_4, t'_{23})\}$;
- $\alpha'(t'_4) = \text{fly}(\text{?plane}, \text{?loc1})$,
 $\alpha'(t'_{21}) = \text{fly}(\text{?pkg}, \text{?plane}, \text{?loc1})$, etc.

The completion profile actually completes the DT: the inserted tasks are connected with their corresponding inner nodes as their children. When we add new nodes into the DT, the integrity of ordering constraints will be destroyed. To avoid that, we define an operator closure to complete the ordering constraints. Formally, for a tree $\mathcal{T} = (T, E)$, we define its closure on the ordering constraint $\prec$ as $\text{closure}(T, E, \prec)$, given by:

$$\prec \cup \bigcup_{t \in T} \{(t', ch), (ch, t'') | ch \in E(t), t' \prec t, t \prec t''\}.$$

Intuitively, the closure operation completes the ordering constraints about the children which should be inherited from their parent.

Next, we define the completion of DT $\mathcal{T}$ by completion profile ρ w.r.t. TIHTN plan σ as $\mathcal{T}_\rho = (T', E', \prec', \alpha', \beta')$, which is given by:

$$\begin{aligned} T' &:= T \cup \bigcup_{t \in T(\rho)} T_\rho^t \\ E' &:= E \cup \{(t, st) | t \in T, st \in T(\text{tn}_\rho^t)\} \\ \prec' &:= \text{closure}(T', E', \prec) \cup \bigcup_{t \in T(\rho)} \ll_{|T_\rho^t \cup E(t)} \\ \alpha' &:= \alpha \cup \alpha_\sigma \\ \beta' &:= (\beta \setminus \{(t, m) | t \in T(\rho)\}) \cup \{(t, m_\rho^t) | t \in T(\rho)\} \end{aligned}$$

where $T(\rho)$ is the set of the inner nodes associated by ρ .

The procedure of completing a DT consists of first connecting the inserted tasks with the inner nodes, then completing the ordering constraints and finally updating the method applied as the refined method. The DT being completed will satisfy the instance:

Proposition 1. Given an HTN problem $\mathcal{P} = (\mathcal{D}, s_0, t_0)$, let σ be one of its TIHTN plans, $\mathcal{T}$ be its corresponding DT and ρ be one of their completion profiles. Then the completed DT $\mathcal{T}_\rho$ satisfies (s_0, t_0) under the new domain $\mathcal{D} + \mathcal{M}_\rho$.

Proof. First, we show that $\mathcal{T}_\rho$ is a valid DT w.r.t. $\mathcal{D} + \mathcal{M}_\rho$. For every node t in $\mathcal{T}_\rho$ with $\beta'(t) = (c, \text{tn}_\rho^t)$, i) the function α is not reduced, so $\alpha'(t) = c$; ii) the edges between the task t and its inserted tasks T_ρ^t are added, so the task network induced by its children is a grounding of m_ρ^t ; iii) $\text{closure}(T', E', \prec)$ guarantees that all ordering constraints of t are propagated to the inserted tasks and $\ll_{|T_\rho^t \cup E(t)}$ only introduces the ordering constraints among the subtasks in the

same method, so conditions 3. and 4. are satisfied; iv) as the completion profile guarantees that no contradict pair about $\ll$ is introduced, condition 5. is satisfied.

Without removing nodes, the root of $\mathcal{T}_\rho$ is still t_0 . As the plan $\vartheta(\mathcal{T}_\rho)$ is the TIHTN plan σ executable in s_0 , $\mathcal{T}_\rho$ satisfies the instance (s_0, t_0) . $\square$

When an HTN problem has incomplete methods, the completion profile offers a way to improve the HTN domain:

Theorem 1. If an HTN problem $\mathcal{P} = (\mathcal{D}, s_0, t_0)$ has a TIHTN plan but no solution, then there is a completion profile ρ where the HTN problem $\mathcal{P}' = (\mathcal{D} + \mathcal{M}_\rho, s_0, t_0)$ is solvable.

Proof. Straightforward. $\square$

Prioritized Preferences

To formalize the phenomenon that the missing of subtasks happens more likely on some methods than other methods, we consider a prioritization on the methods.

Given a method set $\mathcal{M}$, we define a prioritization as a partition on it: $P = \langle P_1, \dots, P_n \rangle$ where $\bigcup_{1 \leq j \leq n} P_j = \mathcal{M}$. Intuitively, the HTN methods in P_i have a higher priority to be refined than those in P_j if $i > j$. We further consider the prioritized preference $\leq_P$ in terms of cardinality: for $\mathcal{M}_1, \mathcal{M}_2 \subseteq \mathcal{M}$, if there is some $1 \leq i \leq n$ such that $|\mathcal{M}_1 \cap P_i| \leq |\mathcal{M}_2 \cap P_i|$ and that $\forall 1 \leq j < i, |\mathcal{M}_1 \cap P_j| = |\mathcal{M}_2 \cap P_j|$, then we write $\mathcal{M}_1 \leq_P \mathcal{M}_2$. We say $\mathcal{M}_1$ is strictly preferred over $\mathcal{M}_2$ w.r.t. P , written by $\mathcal{M}_1 <_P \mathcal{M}_2$, if $\mathcal{M}_1 \leq_P \mathcal{M}_2$ and $\mathcal{M}_2 \not\leq_P \mathcal{M}_1$.

Generally, the prioritization comes from the confidences of domain experts on methods: the method believed to lack subtasks more likely to have a higher priority to be refined. In particular, there exists a class of HTN domains where actions can be stratified according to the decomposition hierarchy ($?$; $?$). We found an interesting phenomenon that the more detailed tasks are more sensitive to these situations and more easily to be thoughtless. In this case, we assume that an action is more abstract than its subtasks and we consider a preference in terms of a stratum-based prioritization: the more abstract actions have a lower priority to be refined.

Preferred Completion Profiles

Generally, we hope to find a completion profile changing the original methods minimally under the prioritized preference.

We first define some notations: for a refined method m_ρ^t , we use $\tau(m_\rho^t)$ to denote its original method m . For a refined method set $\mathcal{M}'$, we use $\tau(\mathcal{M}')$ to denote all the original methods of the refined methods in $\mathcal{M}'$, i.e., $\tau(\mathcal{M}') = \{m \in \mathcal{M} | m = \tau(m'), m' \in \mathcal{M}'\}$. Note that several completions may be associated with the same HTN method. For two HTN methods m'_1 and m'_2 , if $\tau(m'_1) = \tau(m'_2)$, we say m'_1 and m'_2 are homologous.

Definition 3. Given a TIHTN plan and its corresponding DT, a completion profile ρ is preferred w.r.t. preference P if there is not a completion profile ρ' s.t. $\tau(\mathcal{M}_{\rho'}) <_P \tau(\mathcal{M}_\rho)$.

Intuitively, the preferred completion profile refines methods minimally under the prioritized preference.

Next, we will show how to find the preferred completion profile, as shown in Algorithm 1. First, we consider all inserted tasks in the plan as unlabelled (line 1). Then we scan all inner nodes from the nodes with a method of higher priority to the nodes with a method of lower priority (line 2-3). Next, for an inner node, we find the set of candidate subtasks Δ_t from the inserted tasks, which do not violate the ordering constraints if they were inserted as its subtasks (line 5). More specifically, for the inner node t , the inserted tasks which are executed between the last task required to be executed ahead of t and the first task required to be after t , are allowed to be added as subtasks of t . According to the total order ' $\ll$ ' in the DT, we define the subtasks candidate set Δ_t of t as the set of the unlabelled inserted tasks between the last predecessor of t and the first successor of t . Finally, we associate all tasks in the subtask candidate set to t (line 5) and label them as subtasks (line 6). When all inserted tasks are labeled, it returns a preferred completion profile. It must terminate and the worst case is that the inserted tasks are associated with the root task.

Algorithm 1 only scans the inner nodes of the DT once and searching the subtask candidate set can be done in linear time, so the algorithm terminates in polynomial time.

Algorithm 1: COMPLETE($\sigma, \mathcal{T}, P$)

input : A TIHTN plan σ , its DT $\mathcal{T}$ and a prioritization $P = (P_1, \dots, P_n)$ on $\mathcal{M}$
output: A completion profile ρ

```

1  $I \leftarrow I_\sigma;$ 
2 for  $j \leftarrow n$  to 1 do
3   for each  $t \in \mathcal{T}_\tau$  s.t.  $\beta(t) \in P_j$  do
4     if  $I \neq \emptyset$  then
5       for every  $t' \in \Delta_t \cap I$ , set  $\rho(t') = t$ ;
6        $I \leftarrow I \setminus \Delta_t;$ 
7 return  $\rho$ 

```

Actually, to find a preferred completion profile, we only need to scan the inner nodes in the DT according to the preference and link appropriate inserted tasks with inner nodes, which can be done in polynomial time.

Refining Methods from Instances

As stated above, we focus on keeping the original methods coming from domain experts and consider adding the refined methods into the original domain. For an HTN domain $\mathcal{D} = (\mathcal{L}, \mathcal{O}, \mathcal{C}, \mathcal{M})$ and a method set $\mathcal{M}'$, we use $\mathcal{D} + \mathcal{M}' = (\mathcal{L}, \mathcal{O}, \mathcal{C}, \mathcal{M} \cup \mathcal{M}')$ to denote the resulting domain by adding $\mathcal{M}'$ into $\mathcal{D}$. An HTN method refining problem is defined as a tuple $(\mathcal{D}, \mathcal{I})$ where $\mathcal{D}$ is an HTN domain and $\mathcal{I}$ is a set of instances.

However, an excess of methods introduced may slow down problem-solving significantly, as there are excessive choices to decompose tasks. So, we hope the refined methods to be as minimal as possible. Then we define a solution of the HTN method refining problem $(\mathcal{D}, \mathcal{I})$ is a set of refined methods $\mathcal{M}'$ which should satisfy:

- all instances in the set $\mathcal{I}$ are solvable under $\mathcal{D} + \mathcal{M}'$;
- the refined method set $\mathcal{M}'$ is as minimal as possible;
- the refined methods in $\mathcal{M}'$ have as few inserted subtasks as possible.

Given a set of HTN instances, we first compute a TIHTN plan and a DT for each instance and then compute the preferred completion profile. Indeed, the completion profiles for various instances induce many different refined methods with the same head which possibly handel the same situation. Such refined methods are redundant because they can be replaced by other methods.

To compute the minimal set, we define a method substitution operation for a decomposition tree. More specifically, for a DT $\mathcal{T}$ and two homologous refined methods m'_1 and m'_2 , we use $\text{sub}(\mathcal{T}, m'_1, m'_2)$ to denote the resulting DT obtained by replacing every subtree induced by m'_1 with a subtree by m'_2 and completing the closure of the ordering constraints. Then for an instance (s_0, t_0) , if $\mathcal{T}$ satisfies it and $\vartheta(\text{sub}(\mathcal{T}, m'_1, m'_2))$ is executable in s_0 , then $\text{sub}(\mathcal{T}, m'_1, m'_2)$ also satisfies it. In other words, for this instance, the refined method m'_1 is replaceable by m'_2 .

Then we generalize the method substitution operation into sets: given a DT set $\overline{\mathcal{T}}$ and two refined method sets $\mathcal{M}'_1, \mathcal{M}'_2$, we use $\text{sub}(\overline{\mathcal{T}}, \mathcal{M}'_1, \mathcal{M}'_2)$ to denote the set of the DTs that substitutes every method in $\mathcal{M}'_1$ with certain method in $\mathcal{M}'_2$. If each resulting DT still satisfies the corresponding instance, we say $\mathcal{M}'_1$ is replaceable by $\mathcal{M}'_2$. Given a set $\mathcal{M}'$ of refined methods, for the minimality, we need to compute the minimal subset $\mathcal{M}''$ of $\mathcal{M}'$ which are not replaceable by any strict subset of $\mathcal{M}''$.

Indeed, it is difficult to find the minimal method set under the replaceability relation between refined methods, as all subsets of the refined method set require to be considered. Fortunately, the prioritized preference indicates what methods should be refined in a higher priority. The prioritized preference can be extended to the refined methods: $m' \in P_j$ if $\tau(m') \in P_j$. Then we seek for the sub-optimal solution by computing the minimal subset *w.r.t.* the replaceability relation under the refined methods with the same priority, which reduces the searching space significantly.

Next, we give an algorithm to refine methods for a set of HTN problems and a given prioritization, as shown in Algorithm 2. The framework consists of two main components: the first iteration for refining methods (line 2-8) and the second iteration for reducing refined methods (line 9-11).

We developed the TIHTN planning approach in (?) (noted HPLAN). In order to pursue as few inserted subtasks as possible in refined methods, we exploited a breadth-first strategy to search inserted tasks.

In the first iteration, it first invokes HPLAN to compute TIHTN plan and the corresponding DTs (line 3) and then computes preferred completion profiles (line 4) by COMPLETE. According to these completion profiles, it completes these DTs and constructs a set of refined methods.

In the second iteration, we use a greedy strategy to find the minimal set: the refined methods with lower priority are reduced first, which is the opposite against the procedure of searching the preferred completion profile. Here we use

$P_j[\mathcal{M}']$ to denote the refined methods in $\mathcal{M}'$ with the priority P_j . The algorithm enumerates the power set of $P_j[\mathcal{M}']$ and computes the minimal subset $\mathcal{M}_j'$ of $P_j[\mathcal{M}']$ w.r.t. the replaceability relation under $P_j[\mathcal{M}']$ and the union of the minimal subsets with lower priorities.

Algorithm 2: METHODREFINE($\mathcal{D}, \mathcal{I}, P$)

input : An HTN domain $\mathcal{D}$, an instance set $\mathcal{I}$ and a prioritization $P = (P_1, \dots, P_n)$ on $\mathcal{M}$
output: A set of refined methods $\mathcal{M}''$

- 1 $\mathcal{M}' \leftarrow \mathcal{M}'' \leftarrow \emptyset; \quad \overline{\mathcal{T}} \leftarrow \emptyset;$
- 2 **for** each i in $\mathcal{I}$ **do**
- 3 compute a plan and DT
 $(\sigma^i, \mathcal{T}^i) = \text{HPLAN}(\mathcal{D}, i);$
- 4 $\rho^i = \text{COMPLETE}(\sigma^i, \mathcal{T}^i, P);$
- 5 complete the DT $\mathcal{T}^i$ to $\mathcal{T}_\rho^i$ by ρ^i ;
- 6 $\overline{\mathcal{T}} \leftarrow \overline{\mathcal{T}} \cup \mathcal{T}_\rho^i;$
- 7 construct a new method set $\mathcal{M}_\rho^i$ from ρ^i ;
- 8 $\mathcal{M}' \leftarrow \mathcal{M}' \cup \mathcal{M}_\rho^i;$
- 9 **for** $j \leftarrow 1$ to n **do**
- 10 compute the minimal subset $\mathcal{M}_j'$ w.r.t. the replaceability relation under $P_j[\mathcal{M}'] \cup \mathcal{M}''$;
- 11 $\mathcal{M}'' \leftarrow \mathcal{M}'' \cup \mathcal{M}_j'$;
- 12 **return** $\mathcal{M}''$

In fact, due to the greedy strategy, our approach cannot guarantee criterion 2 and 3, but must satisfy criterion 1:

Theorem 2. Suppose $\mathcal{M}''$ is the method set refined by METHODREFINE($\mathcal{D}, \mathcal{I}, P$), if every instance in $\mathcal{I}$ has a TIHTN plan under the domain $\mathcal{D}$, then it is also solvable under the domain $\mathcal{D} + \mathcal{M}'$.

Proof. As every instance has a TIHTN plan, by Proposition 1, there exists a set of DTs $\overline{\mathcal{T}}$, each of which satisfies each instance w.r.t. the domain $\mathcal{D} + \mathcal{M}'$ where $\mathcal{M}'$ is a method set obtained via completion profiles. As in the second iteration, the algorithm keeps the satisfiability of the instances, $\mathcal{M}'$ is replaceable by $\mathcal{M}''$ and each DT in $\text{sub}(\overline{\mathcal{T}}, \mathcal{M}'', \mathcal{M}')$ satisfies its corresponding instance in $\mathcal{I}$. Thus, every instance is solvable w.r.t. the new domain $\mathcal{D} + \mathcal{M}'$. $\square$

Experimental Analysis

In this section, we evaluate METHODREFINE¹ in three well-known planning domains comparing with HTN-MAKER² on the ability of solving new instances.

We consider the three domains which HTN-MAKER uses: Logistics, Satellite, and Blocks-world. We first get the problem generators from International Planning Competition website³ and randomly generate 100 instances for each domain and take 50 instances as the training set and 50 instances as the testing set. We run METHODREFINE

and HTN-MAKER with 50 instances growingly as input and obtain different learned method sets from these two approaches. A testing instance is considered as solved, if its goal is achieved by a plan computed under the learned HTN method set via an HTN planner. For HTN-MAKER we use the HTN planner from their website² and for our approach METHODREFINE we still use our TIHTN planner with forbidding task insertion. Experiments are run on the 3.00 GHz Intel i5-7400 with 8 GB RAM with a cutoff time of one hour. In order to check if an instance is solved, we add a verifying action whose precondition is the goal and whose effect is empty in the last subtask of the initial task. The learning performance is measured via the proportion of the solved instances on the testing instances, which is called *solving rate*.

Experimental results on comparing different domain incompleteness. First, we evaluate the influence of the different incompleteness of the given method sets on the solving rate. To simulate the incomplete method set as the input of METHODREFINE, we take the HTN domain descriptions in the website⁴ of SHOP2 HTN planner, and remove different sets of subtasks from these domains. Then we consider three removal cases: 1) remove one primitive task from each method (if exists), with meaning the high completeness, noted by MR-H; 2) remove two primitive tasks from each method (if exists), noted by MR-M, with meaning the middle completeness; 3) remove one more compound task in some method of MR-L, noted by MR-L, with meaning the low completeness. Taking the method set shown in Figure 1 as example, for MR-H, we remove the first *drive* and the first *fly* in the methods *cityShip* and *airShip*, respectively, while for MR-M, we remove all *drive* and *fly* in the methods. For MR-L, the first *cityShip* is additionally removed from the method of *ship* based on the MR-M setting. As these domain are stratifiable, we use the stratum-based prioritized preference as the input of METHODREFINE.

The experimental results are shown in Figure 2. It demonstrates that the more complete the domain is, the better the learning performance is. Generally, the solving rate increases along with the training set growing. For the Logistics and Satellite domains, in the settings of MR-H and MR-M, METHODREFINE learns the necessary methods to solve all testing problems from a few instances. It is because the structure of these two domains is relatively straightforward and the DTs still can be constructed by the incomplete method sets. In the MR-L setting, the compound action removed in the Logistics Domain, *cityShip*, contains more arguments, making the learned methods become more case-specific, which cannot contribute to other instances.

MR-H eventually learns 2 methods which already effectively solve all testing instances on the three domains. While MR-M and MR-L learn more than 10 methods in the Blocks-World domain but still fail to achieve the full solving rate.

Experimental results on comparing METHODREFINE against HTN-MAKER. From Figure 2, we observe that HTN-MAKER learns methods less slowly than METHODREFINE with MR-H setting. Comparing with the other two settings, HTN-MAKER is superior on the Blocks-World do-

¹ Available in <https://github.com/sysulic/MethodRefine>

² <http://www.cse.lehigh.edu/InSyTe/HTN-MAKER/>

³ <http://ipc02.icaps-conference.org/>

⁴ <https://www.cs.umd.edu/projects/shop/>

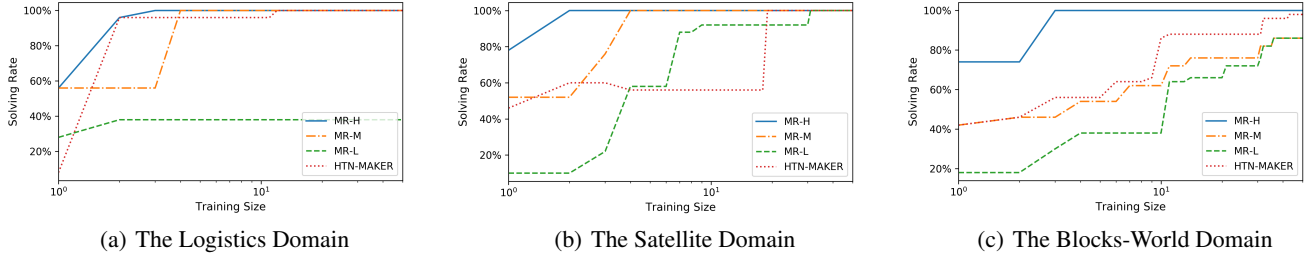


Figure 2: Experimental Results on the Solving Rate of METHODREFINE with Different Domain Incompleteness and HTN-MAKER

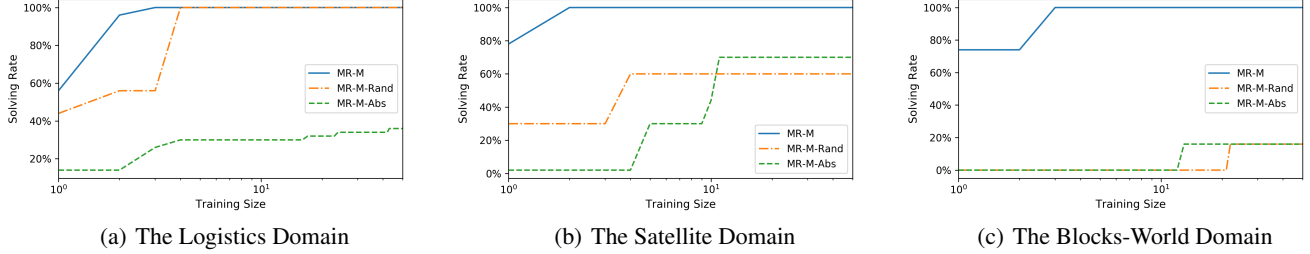


Figure 3: Experimental Results on the Solving Rate of METHODREFINE with Different Preferences

main but inferior on the Logistics and Satellite domain. In the Satellite domain, after training a number of instances, HTN-MAKER exceeds memory limitation when solving some instances, causing a sudden drop in its curve. It is because HTN-MAKER learns a method which causes an infinitely recursive decomposition. It never occurs in our approach because we only learn methods from acyclic decomposition trees. But they are solved when suitable methods are learned with the training set growing. HTN-MAKER finally learns 87, 23, 92 methods in the Logistics, Satellite, Blocks-World domain, respectively. Comparing with our approach, it learns more methods but many of which are redundant.

Experimental results on the performances of METHODREFINE with different preferences. To evaluate our assumption on the stratum-based prioritized preference, we also compare it against its opposite prioritization where tasks are inserted in as abstract compound tasks as possible ('MR-M-Abs'), and the case with no preference where any completion profile is allowed ('MR-M-Rand'). We choose the domains with the middle completeness.

From the experimental results (Figure 3), our stratum-based prioritized preference outperforms the other two cases. It is because the completion profile associates the inserted tasks to a more abstract task and it generates a more case-specific method which may not suit other instances.

Discussion and Conclusion

Without declarative goals, we suppose that in the original method set, every compound action at least has a method to decompose. Our approach can also accept a declarative goal: we can trivially introduce a compound action of achieving the goal which is decomposed into a "verifying" action

whose precondition is the goal and whose effect is empty.

To sum up, we present a framework to help domain experts to improve HTN domains by refining methods. The experiment results demonstrate that our approach outperforms the method learning approach, HTN-MAKER, given an appropriately incomplete method set as input. It is also illustrated that the stratum-based prioritized preference is effective to solve new instances.

In this paper, we assume primitive action models are available. In the future, it would be interesting to remove this assumption and study learning HTN methods and action models simultaneously from history data (?; ?). Also, in this paper we only focus on keeping the procedural knowledge from domain experts and it would be a promising avenue to jointly consider partial annotations of tasks or state constraints together (?; ?). In what follows, we are going to study refining methods directly from raw data, such as texts (?) and to explore plan recognition techniques with incomplete action models (?) to improve HTN domains.

Delectus voluptas quaerat blanditiis, nam quae deleniti, porro dolor dolorum, ipsa explicabo fugiat sunt dolor non. Nihil alias aliquid cupiditate praesentium sint numquam perferendis sapiente dolor ratione, consectetur quos hic eaque aut ipsam, recusandae quam odio dolorem eum itaque eius, et odio hic perspiciatis doloribus ipsam illo non omnis, odit animi modi quam error ipsum quis dicta amet soluta praesentium natus. Fugit dolorum impedit voluptatibus officii cupiditate, earum facilis voluptate eveniet doloremque laudantium nobis, praesentium iure reiciendis qui quibusdam alias nulla sed debitis aut, provident dolorum obcaecati molestias asperiores rerum ipsam incidunt nemo, modi aliquid magnam possimus qui ea officia voluptates mollitia prae-

sentium atque?Eaque facere culpa temporibus itaque ullam
a iusto magnam eos voluptas quam, autem illum perferendis
maiores labore qui, nesciunt voluptatum odio